PA5_Theory

1. Static scoping determines a variable's value following the lexical parent chain, looking in the local scope first, then the parent scope until the declaration is found. Dynamic scoping determines a variable's value by searching the active call stack, starting with the function most recently called and moving backwards from there. Python uses static while Bash uses dynamic. Static scoping is better in a large codebase because you can determine the value of a variable simply by following the source code. With dynamic scoping in a large database, the variable would depend on the active call stack at runtime, which could vary. This can make variables harder to predict, so static scoping is more predictable in this situation.

2. Static: B() --> global x = 10
   Dynamic: B() --> A() local x = 1 -X-> global x = 10 (Doesn't reach here)

x = 10 for static, x = 1 for dynamic, and they diverge at the first step where static scoping goes to the parent global, while dynamic goes to the most recent caller A().

3. A scope hole is when a variable in a function shadows a variable with the same name in an outer function or scope. So, the inner function can no longer access the outer variable, even though the outer variable is still in scope.

Pseudocode example:

```
x = 10

def outer():
    x = 20

    def inner(x):
        print(x)

    inner(5)
```

4. Dynamic scoping can be dangerous in a multi-threaded interpreter because the call stack is shared state if multiple threads use the same stack. If two threads are pushing and popping frames simultaneously, one thread could mess with the other. This could cause the dynamic scoping to search the wrong frames and return an incorrect variable, depending on the active call stack.